

# Backend/API-Option: Aufmerksamkeits-Validierung (Stufe 1+)

---

*Ausarbeitung, wie die aktuelle Browser-Heuristik ( `src/lib/saliency.ts` ) auf **belastbare ~92–96 % Eye-Tracking-Genauigkeit** gehoben wird – mit konkreten Vorgehensoptionen, Ressourcen und potenziellem Geschäftswert.*

---

## 0. Ausgangslage & Engpass

---

- Frontend ist **statischer Export** (GitHub Pages) → kann selbst keine ML-Modelle serverseitig rechnen.
- Die aktuelle Stufe-1-Heuristik ist eine **erste Näherung** (Kontrast + Farbe + Center-Bias), kein trainiertes Modell.
- Für echte Genauigkeit braucht es **ein trainiertes Saliency-Modell** – entweder selbst gehostet oder als API.

**Architektur-Grundmuster (für alle Optionen gleich):**

```
[ Statisches Frontend (GitHub Pages) ]
  | POST /saliency (Bild als Base64/Multipart)
  ▼
[ Validierungs-Endpoint (HTTPS, CORS) ]
  | Inferenz
  ▼
[ Saliency-Modell ] → Heatmap-PNG + Metriken (JSON)
```

Das Frontend bleibt statisch; nur ein **externer Endpoint** kommt hinzu. Die Browser-Heuristik bleibt als **Offline-Fallback** erhalten.

---

# 1. Die drei Optionen im Überblick

|                 | A — Kommerzielle API    | B — Self-hosted DeepGaze        | C — Hybrid          |
|-----------------|-------------------------|---------------------------------|---------------------|
| Genauigkeit     | 92–96 % (MIT-validiert) | ~SOTA (DeepGaze IIE)            | A jetzt → B später  |
| Time-to-Market  | <b>Tage</b>             | 2–4 Wochen                      | Tage, dann iterativ |
| Fixkosten/Monat | hoch (Abo)              | niedrig–mittel (Infra)          | startet niedrig     |
| Wartung         | ~0 (Anbieter)           | laufend (DevOps)                | mittel              |
| Datenhoheit     | beim Anbieter           | <b>vollständig eigen</b>        | wandert zu eigen    |
| Lizenzrisiko    | gering (kommerziell)    | <b>prüfen</b> (Forschungscodes) | gemanagt            |

## 2. Option A — Kommerzielle API (schnellster Weg)

### Anbieter

- **Attention Insight API** — predictive Eye-Tracking, am MIT/Tübingen-Benchmark **92,5–96 %** validiert. Developer-API + fertige Figma-/Adobe-/Chrome-Plugins. Preis: Team-Plan **~479 €/Monat** (≈500 Credits/Monat). API-Zugang in höheren Tarifen.
- **Neurons „Predict“ API** — >95 % Genauigkeit, zusätzlich Metriken wie *Attention*, *Intent*, *Trust*, *Cognitive Demand*. Enterprise: **~15.000 €/Jahr** (5 Seats inkl. API).

### Umsetzung

1. API-Key besorgen (Trial vorhanden).
2. **Dünnere Proxy** (eine Serverless-Funktion), der den Key versteckt und CORS löst — z.B. Vercel/Netlify Funktion oder Cloudflare Worker. **Niemals den Key ins statische Frontend legen.**
3. `Validation.tsx` : bei „Analysieren“ statt lokaler Heuristik den Proxy aufrufen, Heatmap + Focus-Score rendern.

Frontend → Cloudflare Worker (hält API-Key) → Attention Insight API → Heatmap+Score

### Ressourcen

- **Dev:** ~1–2 Tage (Proxy + Frontend-Anbindung).
- **Infra:** Worker/Funktion ~0–5 €/Monat; **API-Abo 479 €/Mo bzw. 15 k €/Jahr.**
- **Wartung:** quasi null.

## Win

- Sofort **vertriebsfähige, zitierfähige Genauigkeit** (»MIT-validiert«).
- Kein ML-Know-how nötig, kein Modell-Betrieb.
- Risiko: laufende Stückkosten, Abhängigkeit, Daten verlassen das Haus.

---

## 3. Option B — Self-hosted DeepGaze (volle Kontrolle)

---

### Modell

- `matthias-k/DeepGaze` (PyTorch): DeepGaze **IIE** (räumliche Saliency-Heatmap) und **III** (Scanpath/Blickreihenfolge). Vortrainierte Gewichte verfügbar (MIT1003/SALICON), Center-Bias-Datei inklusive.
- **⚠ Lizenz-Due-Diligence nötig:** Repo ist Forschungscode, Gewichte primär für Forschung. **Vor kommerziellem Einsatz Lizenz klären** (Autoren kontaktieren / eigenes Training auf offenem SALICON / oder kommerzielle Alternative). Das ist der wichtigste Show-Stopper-Check für diese Option.

### Deployment-Varianten

| Variante                   | Beschreibung                                                       | Kosten-Profil            | Latenz           |
|----------------------------|--------------------------------------------------------------------|--------------------------|------------------|
| <b>B1 — CPU-Container</b>  | DeepGaze IIE auf CPU (Cloud Run / Fly.io / Render), Scale-to-Zero  | ~0 € idle,<br>~Cent/Bild | ~1–3 s/Bild      |
| <b>B2 — Serverless GPU</b> | Modal/Replicate, <code>@app.function(gpu=...)</code> , pay-per-use | Cold-Start, ~Cent-€/Bild | <1 s warm        |
| <b>B3 — Dedizierte GPU</b> | RunPod/Lambda Dauer-GPU                                            | ~150–600 €/Mo fix        | konstant niedrig |

Für **niedriges/mittleres Volumen** ist **B1 (CPU + Scale-to-Zero)** am wirtschaftlichsten: bei Einzelbild-Validierung sind 1–3 s völlig akzeptabel, Leerlauf kostet nichts.

### Umsetzung

1. FastAPI-Service: `POST /saliency` → lädt DeepGaze IIE einmalig, gibt Heatmap-PNG + Metriken (Konzentration, Fokus, AOI-Scores) zurück.
2. Als Container auf **Google Cloud Run** (Scale-to-Zero, HTTPS, CORS out-of-the-box).
3. Frontend ruft den Endpoint auf; Browser-Heuristik bleibt Fallback.

```
# Skizze
model = deepgaze_pytorch.DeepGazeIIE(pretrained=True).to(DEVICE)
@app.post("/saliency")
def saliency(file: UploadFile):
    img = preprocess(file)          # downscale auf ~1024px
    logd = model(img_tensor, centerbias) # log-density
    return { "heatmap_png": ..., "metrics": derive(logd) }
```

## Ressourcen

- **Dev:** ~2–4 Wochen (Service, Container, Deployment, Frontend, Lizenzklärung, Tests).
- **Infra:** B1 ~5–30 €/Mo bei moderatem Volumen; GPU-Varianten höher.
- **Wartung:** laufend (Updates, Monitoring, Skalierung) — DevOps-Kompetenz nötig.

## Win

- **Keine Stückkosten pro Bild**, volle **Datenhoheit** (DSGVO-Vorteil ggü. US-APIs).
- Differenzierung: **eigenes Modell** als Teil des PROVOID-IP, frei erweiterbar (z.B. systemspezifische Re-Kalibrierung).
- Risiko: Lizenz, Betriebsaufwand, ML-Verantwortung im Haus.

---

## 4. Option C — Hybrid (empfohlener Pfad)

---

**Jetzt mit API starten, bei Volumen/Marge auf Self-Hosting migrieren.**

1. **Phase 1 (Wochen 0–2):** Option A anbinden → sofort belastbare Genauigkeit, vertriebsfähig. Stückkosten akzeptabel bei kleinem Volumen.
2. **Phase 2 (parallel):** Lizenz für DeepGaze klären / SALICON-Training evaluieren; CPU-Service als PoC bauen.
3. **Phase 3 (ab Schwelle):** Sobald monatliche API-Kosten den Self-Hosting-Aufwand übersteigen (grobe Faustregel: **ab ~1.000–2.000 Bildern/Monat**), auf B1 umschalten. Da das Frontend-Interface gleich bleibt ( `POST /saliency` ), ist der Wechsel **transparent**.

**Vorteil:** minimales Anfangsrisiko, sofortiger Marktwert, sauberer Migrationspfad ohne Frontend-Umbau.

---

## 5. Entscheidungs-Matrix

| Kriterium                      | Gewicht | A (API) | B (Self) | C (Hybrid) |
|--------------------------------|---------|---------|----------|------------|
| Time-to-Market                 | hoch    | •••     | •        | •••        |
| Genauigkeit/Glaubwürdigkeit    | hoch    | •••     | •••      | •••        |
| Laufende Kosten bei Skalierung | mittel  | •       | •••      | •••        |
| Datenhoheit/DSGVO              | mittel  | •       | •••      | •••        |
| Betriebsaufwand                | mittel  | •••     | •        | ••         |
| Lizenz-/Abhängigkeitsrisiko    | mittel  | •••     | •        | ••         |

→ **Empfehlung: Option C (Hybrid)**. Start mit Attention-Insight-API über einen Key-versteckenden Proxy, parallel DeepGaze-PoC + Lizenzklärung.

## 6. Geschäftswert (potenzieller Win)

**Produkt/Vertrieb** - Aus einer „netten Heuristik“ wird ein **zitierfähiges Validierungs-Versprechen** (»MIT-validiert, 95 %«) — schließt den Framework-Loop sichtbar. - Klarer **Upsell-Pfad**: Stufe 1 (KI, Self-Service) → Stufe 2 (Webcam-Panel) → Stufe 3 (EEG-Labor = PROVOID-Kerngeschäft).

**Monetarisierung (Beispiel-Logik, zu validieren)** - Stufe 1 als **Lead-Magnet/Freemium** → Conversion in bezahlte Workshops/Beratung. - Bei API-Stückkosten im **Cent-Bereich/Bild** und Verkauf von Validierungs-Credits/Reports ist die **Marge hoch**; Self-Hosting drückt Grenzkosten Richtung null.

**Strategisch - Datenhoheit** (Self-Hosting) als Verkaufsargument bei DSGVO-sensiblen Kund:innen. - Eigenes Modell = **verteidigbares IP**, koppelbar an die System-Profile ( `SYSTEM_PROFILES` ) für framework-spezifische Auswertung, die generische Tools nicht bieten.

## 7. Konkrete nächste Schritte

1. **Trial-Key** bei Attention Insight ziehen, Genauigkeit an echten Kampagnen-Visuals gegentesten.
2. **Proxy-Function** (Cloudflare Worker) aufsetzen, Key sichern, `Validation.tsx` anbinden (Heuristik bleibt Fallback).
3. Parallel **DeepGaze-Lizenz** klären (Autoren / SALICON-Eigenttraining prüfen).

4. **Schwellenwert** definieren (Bilder/Monat), ab dem Migration auf B1 (Cloud Run, CPU, Scale-to-Zero) wirtschaftlich wird.
  5. Frontend-Interface stabil halten ( POST /saliency ), damit der Backend-Tausch transparent bleibt.
- 

*Quellen: Attention Insight (MIT/Tübingen-Benchmark, API-Doku, Pricing), Neurons API-Doku/Pricing, matthias-k/DeepGaze (PyTorch, IIE/III), Hugging Face / Modal / Cloud Run Deployment-Optionen. Preise sind Richtwerte und vor Vertragsabschluss zu bestätigen.*